

2.1 Hough 与 YOLO 感知方案对比

比较 Hough 圆检测和 YOLO 检测在足球识别任务中的特点。

完成内容：

- 说明 Hough 圆检测的输入、输出和基本思路；
- 说明 YOLO 检测的输入、输出和基本思路；
- 比较两者对光照、背景、遮挡、球颜色和运行速度的敏感性；
- 说明为什么 YOLO Demo 中需要 fallback；
- 分析在 VMware 虚拟机中运行 YOLO 可能遇到的性能问题，并给出解决思路；
- 如果你应用 YOLO 进行目标检测，简要设计一套从数据采集、模型训练到部署运行的方案。

提交内容：

- Hough 与 YOLO 的对比表；
- 两种方案的优缺点分析；
- YOLO 性能分析与方案设计。

2.2 双机器人 namespace 通信分析

运行或分析双机器人 Demo，理解 namespace 在多机器人系统中的作用。

参考命令：

```
roslaunch robot_soccer_integrated_demo two_robot_hough_demo.launch
```

可以使用以下命令查看两台机器人的关键话题：

```
rostopic echo /robot1/decision_state
```

```
rostopic echo /robot1/cmd_vel
```

```
rostopic echo /robot2/goalie_state
```

```
rostopic echo /robot2/cmd_vel
```

完成内容：

- 说明 /robot1 和 /robot2 的角色；
- 列出两台机器人的关键话题；

- 解释为什么多机器人系统必须使用 namespace;
- 画出双机器人通信结构图;
- 分析如果没有 namespace, 可能出现哪些话题混淆或控制冲突。

提交内容:

- 双机器人关键话题表;
- 双机器人通信结构图;
- namespace 作用说明;
- 可能的话题混淆或控制冲突分析。

提高作业提交要求

请将提高作业整理为 一份 PDF 提交。

Answers:

2.1 Hough 与 YOLO 感知方案对比

(1) Hough 与 YOLO 的对比表

对比维度	Hough 圆检测	YOLO 检测
输入	/robot1/camera/image_raw 图像	/robot1/camera/image_raw 图像
输出	发布到/robot1/vision/detections, 字段为 x_error、y_error、width_norm、height_norm、confidence、class_id; 同时可发布 /robot1/hough/debug_image	同样发布到/robot1/vision/detections; 同时可发布/robot1/yolo/debug_image, 并标注 YOLO 或 FALLBACK
基本思路	先用 HSV 阈值提取橙色区域, 再用高斯滤波与 Hough 圆寻找近似圆形目标, 选取得分最高的圆作为足球	使用 yolo11n.pt 对图像做目标检测, 筛选 sports ball 或名称中包含 ball 的类别, 取置信度最高的检测框
光照与背景	对光照、阴影、白平衡和背景中相近颜色比较敏感, 颜色阈值需要根据场景调参	对光照和复杂背景通常更稳健, 但泛化能力取决于训练数据覆盖
遮挡与形变	依赖圆形轮廓, 足球被遮挡、只露出一部分或运动模糊时容易漏检	能利用纹理、边缘和上下文信息, 轻微遮挡下仍可能检测到, 但严重遮挡会降低置信度
球颜色	强依赖橙色 HSV 范围; 如果换成其他颜色球, 需要重新设阈值	不强依赖单一颜色, 只要训练集中覆盖相应外观, 就可以识别不同颜色/花纹的球
运行速度	计算量小, 实验中 Hough 限频约 15Hz, 适合 CPU 和虚拟机环境	推理计算量大, 实验中 YOLO 限频较低; 若调高限频, 在 VMware (无 GPU) 出现卡顿现象
适用场景	适合颜色和形状明确、环境可控、计算资源受限的场景	适合背景复杂、目标类别多、颜色形态变化较大的真实视觉任务

(2) 两种方案的优缺点分析

Hough 方案的优点是实现简单、速度快、可解释性强，能直观看到颜色阈值、圆检测和中心误差之间的关系；缺点是对环境假设较强，主要适用于橙色圆球这种先验明确的目标。当场地光照改变、背景出现橙色干扰、球被遮挡时，误检和漏检都会增加。

YOLO 方案的优点是检测能力更通用，可以通过训练数据学习球的多种外观，在复杂背景和轻微遮挡下通常比 Hough 方案更稳健；缺点是依赖模型、算力，部署比 Hough 更复杂。

(3) 为什么 YOLO Demo 中需要 fallback

代码中 YOLO 检测的目标类别为 sports ball，并设置允许 accept_any_ball_class。当模型没有检测出可接受的球类时，yolo_detector_node 会启用 fallback_color_circle，用与 Hough 相近的橙色圆检测继续发布/robot1/vision/detections。这样的兜底检测保证后续位姿估计和 FSM 决策的连续性。

(4) VMware 中运行 YOLO 的性能问题与解决思路

性能问题：在 VMware 虚拟机中运行 YOLO Demo 时，出现/robot1/yolo/debug_image 刷新慢、决策状态多次出现 RECOVER。

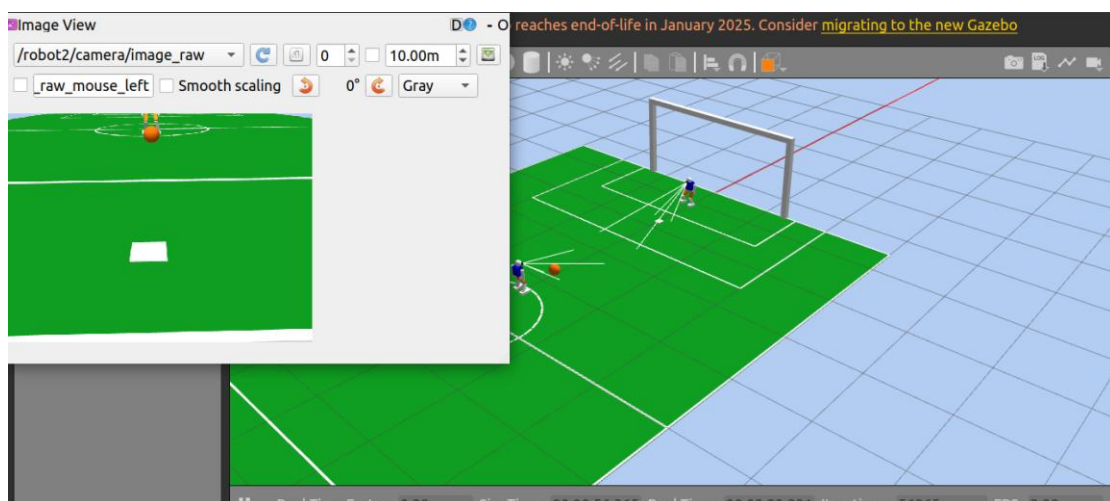
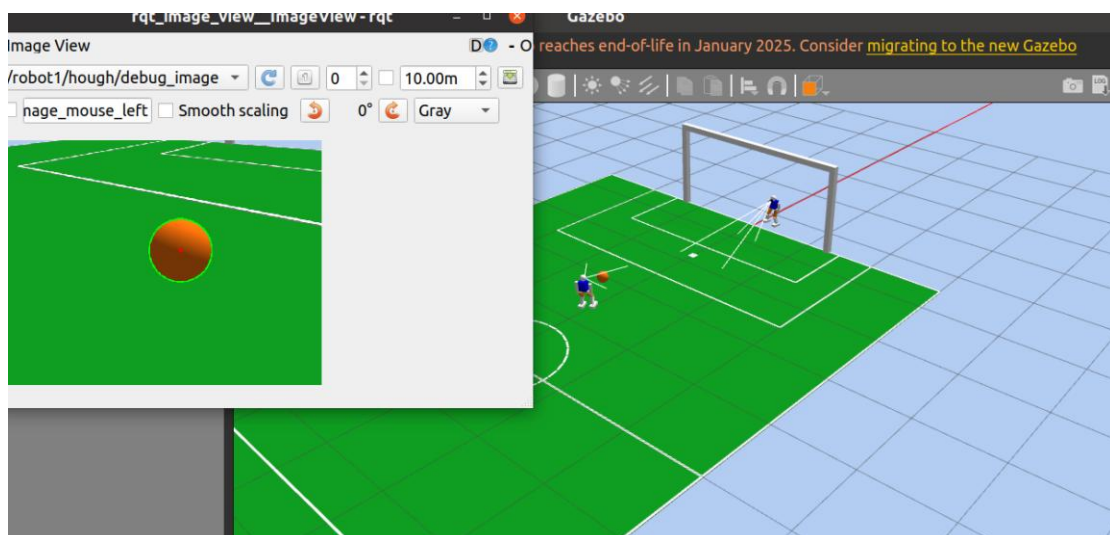
解决思路：主要问题在于虚拟机缺少 GPU 只能进行 CPU 推理且分配给虚拟机的 CPU 核数、内存受限，另外其他进程比如 ROS 图像传输、处理也与模型推理抢占资源。可以尝试降低 imgsz（Demo 中配置为 320）；提高置信度过滤并限制 publish_rate；控制图像队列长度；给虚拟机分配更多 CPU/内存并开启硬件虚拟化等优化方式。

(5) YOLO 目标检测应用方案设计

- 数据采集：利用 Gazebo 相机、真实机器人相机，在不同光照/角度/距离下采集足球图像，覆盖无遮挡、部分遮挡、运动模糊、背景干扰和不同球外观。
- 数据标注：使用矩形框标注 ball/sports ball，划分训练集、验证集和测试集，并检查小目标、边缘目标和困难样本比例。
- 模型训练：以轻量 YOLO 模型为基础进行迁移学习，先用较低分辨率快速验证，再根据召回率和误检率调参；重点关注球类目标的 precision、recall、mAP 和端到端延迟。
- 模型导出与部署：将模型导出为适合运行平台的格式，在 ROS 节点中加载，统一发布/vision/detections，保持与位姿估计和决策节点的接口一致。
- 闭环测试：在仿真和真实场景中测试，记录漏检、误检、延迟和控制抖动，再迭代数据和模型。

2.2 双机器人 namespace 通信分析

(1) 双机器人 Demo 截图



(2) /robot1 和 /robot2 角色说明

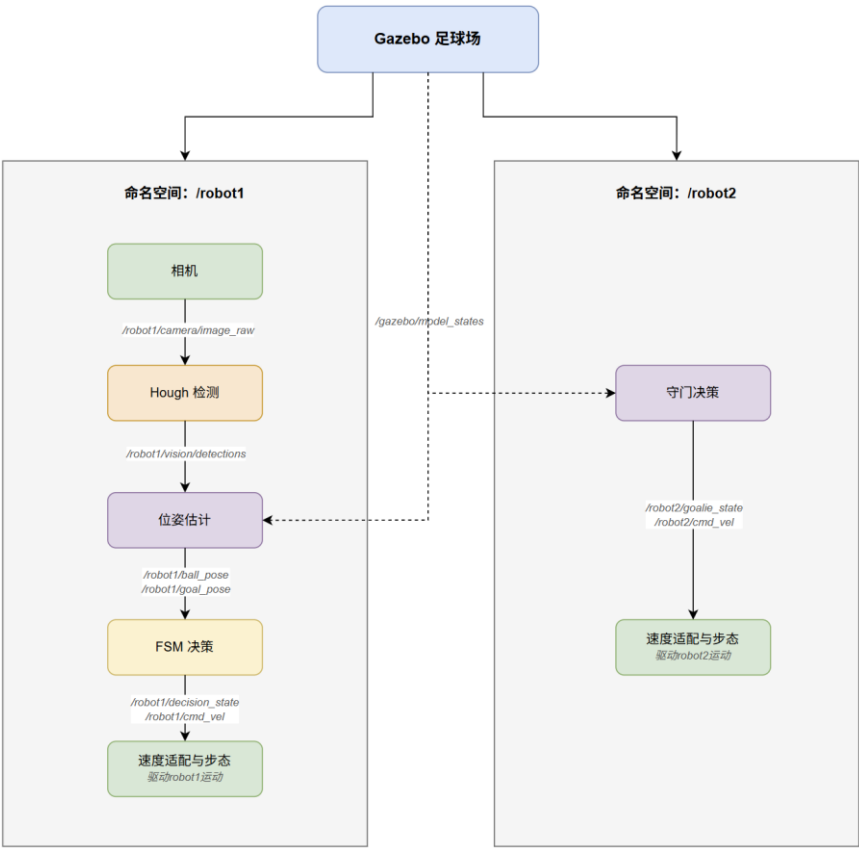
two_robot_hough_demo.launch 中使用/robot1 和/robot2 两个 group。/robot1 是 attacker,

运行 Hough 检测、vision_pose_provider 和 soccer_demo_node，负责寻找足球、接近、带球并射门；/robot2 是 goalie，运行 goalie_node，根据/gazebo/model_states 中足球的 y 坐标在球门前横向防守。

(3) 双机器人关键话题表

对象	关键话题	作用
/robot1 attacker	/robot1/camera/image_raw /robot1/vision/detections /robot1/hough/debug_image	进攻机器人的足球视觉检测
	/robot1/ball_pose /robot1/goal_pose	足球和球门在 robot1/base_link 下的相对位姿
	/robot1/decision_state /robot1/cmd_vel	输出 SEARCH_BALL、APPROACH_BALL、DRIBBLE_BALL、SHOOT 等状态和速度指令
	/robot1/humanoid/position_controller/command	将/robot1/cmd_vel 转为 IKWalk 关节命令
/robot2 goalie	/robot2/goalie_state /robot2/cmd_vel	输出 WAIT_MODEL_STATES、TRACK_BALL_Y、HOLD_GOAL 等守门状态和横向防守速度
	/robot2/humanoid/position_controller/command	将/robot2/cmd_vel 转为 robot2 的步态关节命令
Gazebo	/gazebo/model_states /gazebo/set_model_state	提供全局模型位姿；两台机器人按 robot_model 参数读取自己的模型并更新运动

(4) 双机器人通信结构图



(5) namespace 的作用说明

namespace 的核心作用是给每台机器人提供独立的命名空间,使得话题、节点和控制器在机器人之间相互隔离。比如在 robot1 和 robot2 中都有 cmd_vel、/humanoid/...等话题,但由于在不同 namespace 下,解析后的话题变成/robot1/cmd_vel 和/robot2/cmd_vel、/robot1/humanoid/...和/robot2/humanoid/..., 因此可以复用代码而互不抢占控制通道。

(6) 没有 namespace 时可能出现的话题混淆或控制冲突

1. 两台机器人的 cmd_vel 都叫/cmd_vel 时, 进攻机器人和守门员的速度指令会混到同一个话题, 可能导致 robot1 执行守门动作或 robot2 执行进攻动作。
2. 两个 cmd_vel_adapter 如果都向同一个/humanoid/position_controller/command 发布关节命令, 会互相覆盖步态控制, 造成机器人抖动、停顿或动作不可预测。
3. 相机、检测不区分/robot1 与/robot2 时, 视觉节点可能订阅错相机, 进而把另一台机器人的画面当作自己的感知输入。
4. 两个 robot_state_publisher、控制器和 TF 前缀不区分时, 关节状态和坐标系名称会冲突, 多机器人位姿关系无法可靠维护。